



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unidad 01 — Python básico

Bloque 01

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea



La idea en una frase

“

Python es un lenguaje **interpretado**, de **tipado dinámico** y sintaxis limpia, diseñado para que los programas se lean casi como pseudocódigo, y será el lenguaje con el que trabajaremos todo el módulo de PSP.

¿Qué es Python?

Un lenguaje de programación de alto nivel, interpretado y multiplataforma. Creado por Guido van Rossum en 1991.

Característica	Qué significa en la práctica
Interpretado	No necesitas compilar. Escribes el código y lo ejecutas directamente.
Tipado dinámico	No declares tipos: <code>x = 5</code> y luego <code>x = "hola"</code> funciona sin problemas.
Sintaxis limpia	El código se lee casi como inglés. Las indentaciones obligatorias fuerzan código limpio.
Multiplataforma	Funciona en Windows, Linux y macOS sin cambiar nada.
Biblioteca estándar enorme	<code>os</code> , <code>sys</code> , <code>threading</code> , <code>socket</code> , <code>json</code> ... todo incluido.
Comunidad masiva	Casi cualquier duda que tengas, ya alguien la respondió en Stack Overflow.

| 🤔 ¿Por qué Python para PSP?

En este módulo vamos a crear procesos, hilos, servidores, clientes y APIs:

- `subprocess` y `threading` están en la biblioteca estándar: no necesitas instalar nada.
- `socket` permite crear servidores TCP y UDP en pocas líneas.
- `requests` y `httpx` hacen peticiones HTTP fáciles.
- `asyncio` permite programación asíncrona para servidores concurrentes.
- La sintaxis clara reduce errores cuando estás aprendiendo conceptos nuevos.



Instalar Python

Windows

1. Ve a python.org/downloads
2. Descarga la última versión (3.10 o superior)
3. **Importante:** marca la casilla *"Add Python to PATH"*
4. Verifica en terminal:

```
python --version  
# Python 3.11.5
```



En Windows, si no marcas "Add Python to PATH", la terminal no reconoce el comando `python`. Tendrías que usar la ruta completa o añadirlo manualmente.

Linux (Ubuntu / Debian)

```
sudo apt update  
sudo apt install python3 python3-pip  
python3 --version
```

macOS

```
brew install python3  
python3 --version
```



Entorno virtual (`venv`)

Un proyecto, sus dependencias

Un **entorno virtual** es una carpeta aislada con su propia instalación de Python y sus propios paquetes, separada del sistema global.

Ventaja Principal	Qué soluciona en la práctica
Aislamiento	Los paquetes solo existen en ese entorno, sin contaminar el sistema base.
Reproducibilidad	Con <code>pip freeze > requirements.txt</code> recreas el entorno exacto en otro equipo.
No rompes nada	Si una librería falla en un proyecto, los demás no se ven afectados.



Consejo profesional: crea siempre un entorno virtual antes de empezar. La carpeta `venv/` va al `.gitignore` : no se sube al repositorio.



Operativa paso a paso de venv

Fase / Acción	Windows	Linux / macOS
1. Crear entorno <i>(una vez por proyecto)</i>	<code>python -m venv venv</code>	<code>python3 -m venv venv</code>
2. Activar entorno	<code>venv\Scripts\activate</code>	<code>source venv/bin/activate</code>
3. Verificar activación	Deberías ver el prefijo <code>(venv)</code> al inicio del prompt. Comprueba con <code>python --version</code> .	Lo mismo
4. Desactivar al acabar	Escribe simplemente <code>deactivate</code> en la terminal.	Lo mismo



Cuando el entorno está activo, el prompt muestra el prefijo `(venv)`.

Tu primer programa

Abre un editor de texto (VS Code, PyCharm o incluso el Bloc de notas) y escribe:

```
print("¡Hola, mundo!")
```

Guárdalo como `hola.py` y ejecútalo en la terminal:

```
python hola.py  
# Salida: ¡Hola, mundo!
```



¿Ha funcionado? Enhorabuena: ya tienes Python instalado y sabes ejecutar un programa.

La consola interactiva

Python ofrece una consola interactiva donde puedes probar código línea a línea. Abre una terminal y escribe `python` :

```
>>> 2 + 3
5
>>> "hola".upper()
'HOLA'
>>> exit()
```



La consola interactiva es perfecta para probar cosas rápido antes de meterlas en un archivo `.py` .



Mini-chequeo — Introducción

1. ¿Qué pasa si no marcas *"Add Python to PATH"* al instalar en Windows?
2. ¿Cuál es la diferencia entre el archivo `.py` y la consola interactiva?
3. ¿Python necesita compilar los programas antes de ejecutarlos?

Respuestas:

1. La terminal no reconoce el comando `python`. Tendrías que usar la ruta completa o añadirlo manualmente al PATH.
2. El archivo `.py` es un programa guardado que puedes reejecutar cuantas veces quieras. La consola interactiva es para probar código suelto sin guardar.
3. No. Python es interpretado: las instrucciones se traducen y ejecutan una a una, sin generar un archivo binario intermedio.



Comentarios de línea (#)

Los comentarios explican el código a quien lo lee —incluido tú dentro de tres meses—. En Python se escriben con `#` para una línea y con `"""` para bloques multilinea. Python los ignora por completo al ejecutar.

```
# Este es un comentario completo de línea  
x = 5 # Este comentario va al final de la línea de código
```

Buena Práctica

Separa el comentario del código con al menos un espacio.

Ejemplo Correcto

```
x = 5 # valor  
inicial
```

Ejemplo Incorrecto

```
x = 5 #valor  
inicial
```



Cuándo usar comentarios

- Para explicar **por qué** haces algo, no qué hace el código (eso se lee solo).
- Para dejar una nota temporal: `# TODO: mejorar esto más adelante`.
- Para bloquear código que no quieres ejecutar de momento:

```
# print("Esta línea no se ejecuta")  
x = 10
```

⚠ Un comentario es texto que Python ignora por completo al ejecutar. Sirve para documentar, explicar la lógica o dejar notas. Es la diferencia entre código que funciona y código que funciona **y se entiende**.



Docstrings ("""comentarios multilínea""")

Para comentarios de varias líneas se usan tres comillas dobles `"""`. Esto se llama **docstring** y se usa especialmente para documentar funciones, clases y módulos:

```
"""
Este es un comentario multilínea.
Puede ocupar las líneas que necesites.
Python lo ignora completamente.
"""
```



Docstrings en funciones

Cuando pones un docstring como **primera instrucción** de una función, se convierte en su documentación oficial:

```
def sumar(a, b):  
    """  
    Suma dos números y devuelve el resultado.  
  
    Parámetros:  
        a (int o float): primer número  
        b (int o float): segundo número  
  
    Retorna:  
        int o float: la suma de a y b  
    """  
    return a + b
```

Puedes acceder a ese docstring con `help()` o con `sumar.__doc__`:

```
help(sumar)  
# Salida: Suma dos números y devuelve el resultado...
```



Docstrings en clases

Lo mismo aplica a las clases:

```
class Persona:
    """
    Representa una persona con nombre y edad.

    Atributos:
        nombre (str): nombre de la persona
        edad (int): edad en años
    """

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
```



Comillas simples vs dobles

Python acepta comillas simples `'` y dobles `"` para strings. Para docstrings se usan `"""` (tres comillas dobles) por convención, aunque `'''` (tres comillas simples) también funciona:

```
# Todos son strings válidos
'Hola'
"Hola"
'''Hola'''
"""Hola"""
```



La convención del **PEP 257** (guía de estilo de Python) recomienda `"""` para docstrings.



Errores comunes

Error Común	Explicación y Corrección
Olvidar cerrar comillas	<code>"Hola mundo</code> provoca un <code>SyntaxError: invalid syntax</code> . Revisa que cada <code>"</code> o <code>"""</code> tenga cierre.
Confundir string con docstring	Asignar un bloque <code>mensaje = """Texto"""</code> crea una variable de texto, no un docstring de documentación. Debe ser la primera instrucción de la definición.
Comillas simples vs dobles	Python acepta <code>' '</code> simples, pero la norma oficial PEP 257 recomienda encarecidamente tres comillas dobles <code>"""</code> .



Resumen — Bloque 01

- Python es un lenguaje interpretado, de tipado dinámico y sintaxis limpia, ideal para empezar.
- Para instalarlo, ve a python.org (Windows) o usa el gestor de paquetes (Linux/macOS) y verifica con `python --version`.
- Crea siempre un entorno virtual (`venv`) antes de empezar un proyecto.
- Tu primer programa es un `print("¡Hola, mundo!")` : guárdalo en un `.py` y ejecútalo con `python hola.py`.
- Los comentarios de línea usan `#` y se ignoran al ejecutar.
- Los docstrings usan `"""` y documentan funciones, clases y módulos.
- Comenta **por qué**, no el **qué**: el código debe ser autoexplicativo.



Vocabulario rápido

Término	Idea general
Interpretado	Se ejecuta línea a línea, sin compilar
Tipado dinámico	El tipo de una variable puede cambiar
Alto nivel	Cerca del lenguaje humano
PATH	Dónde busca el sistema los ejecutables
Consola interactiva	Probar código línea a línea
Módulo	Archivo <code>.py</code> con código reutilizable

Término	Idea general
venv	Carpeta aislada con su propio Python y paquetes
Comentario	Texto con <code>#</code> que Python ignora
Docstring	Comentario <code>"""</code> que documenta funciones y clases
PEP 257	Convención para escribir docstrings
Autoexplicativo	Código que se entiende por su nombre

